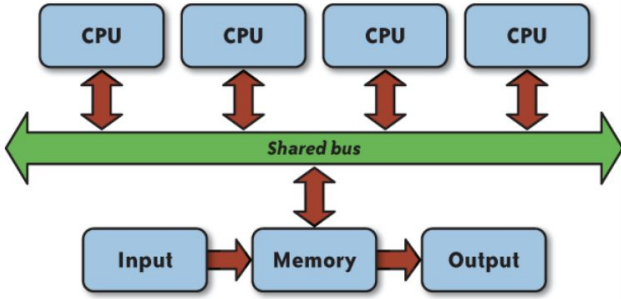
Event-based Programming

Slides partially adapted from Chris Rossbach’s CS378h at UT Austin

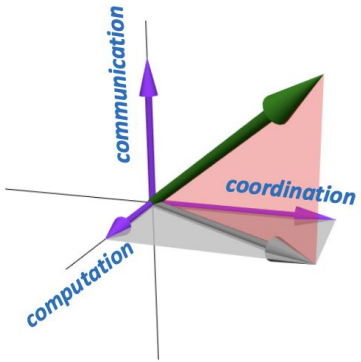
Programming Models for Concurrency

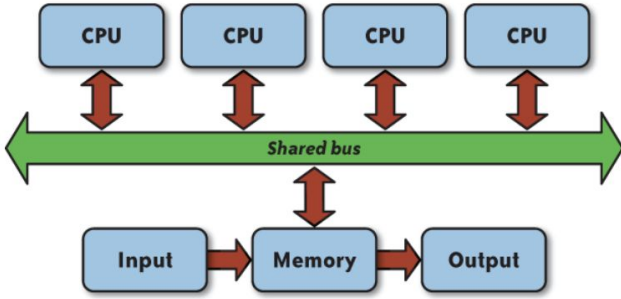
- Hardware execution model
  - CPU(s) execute instructions sequentially, GPUs are a bit different



Programming Models for Concurrency

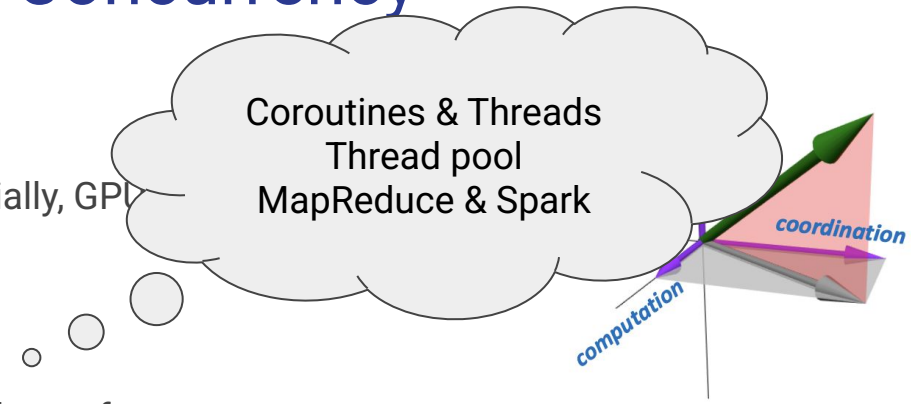
- Hardware execution model
  - CPU(s) execute instructions sequentially, GPUs are a bit different
- Programming model dimensions
  - How to specify computation
  - How to specify communication
  - How to specify coordination / control transfer

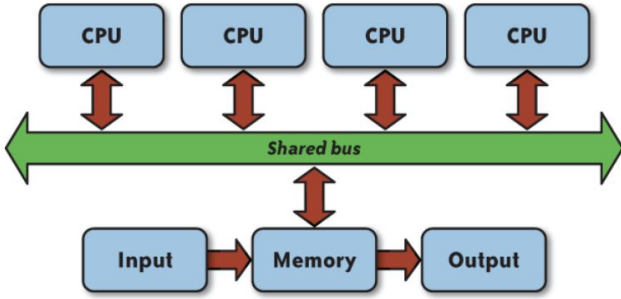




Programming Models for Concurrency

- Hardware execution model
  - CPU(s) execute instructions sequentially, GPU
- Programming model dimensions
  - How to specify computation
  - How to specify communication
  - How to specify coordination / control transfer
- Techniques/primitives
  - Message passing vs shared memory
  - Preemption vs non-preemption
- Dimensions/techniques not always orthogonal





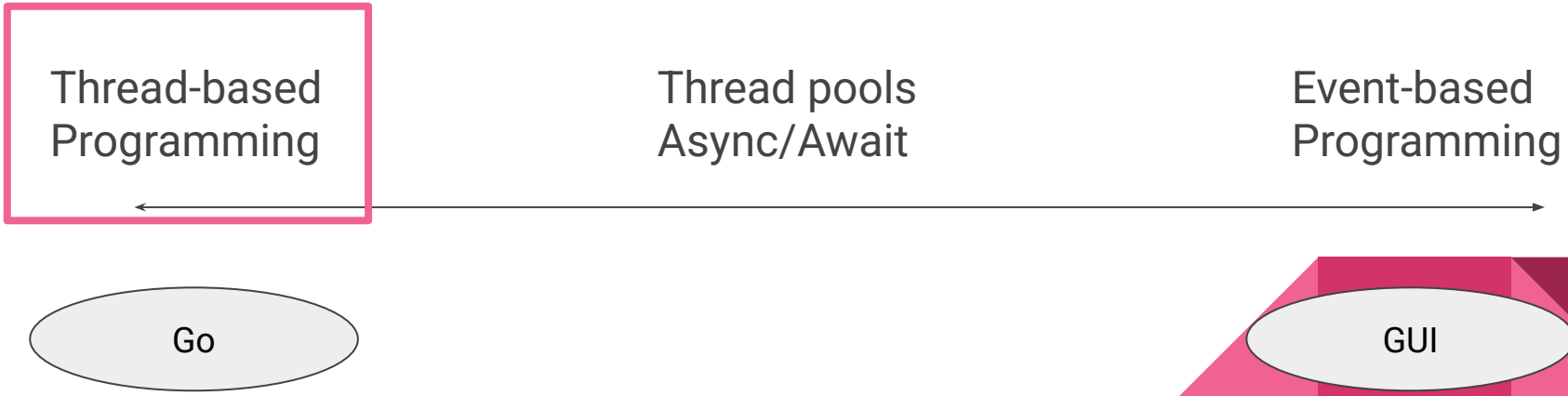
Today’s Axis: The Programming Model

Thread-based Programming

Thread pools Async/Await

Event-based Programming

Today's Axis: The Programming Model



Thread-based: Go

One of the most popular emerging programming languages

Their gimmick is coroutines, which they playfully call “goroutines”

Go

`f(x)` to run f

`go f(x)` to run f in its own goroutine

Go

```go
func main() {
    ch := make(chan int)
    go worker(ch)

    r := <-ch
}
```

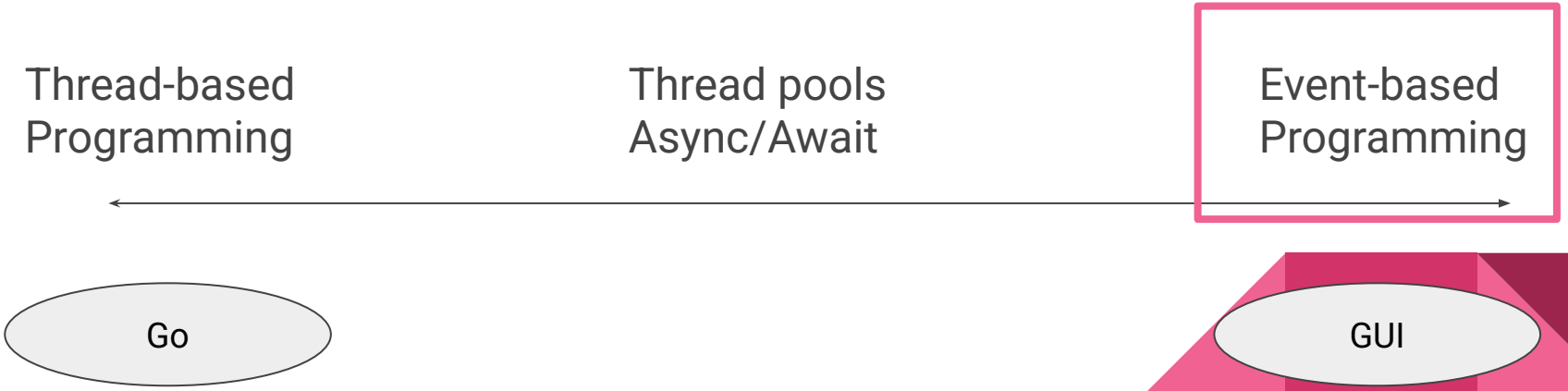
```go
func worker(ch chan int) {
    fmt.Print("working...")
    time.Sleep(time.Second)
    fmt.Println("done")

    ch <- 42
}
```

Go’s runtime

A ready queue

A yield function

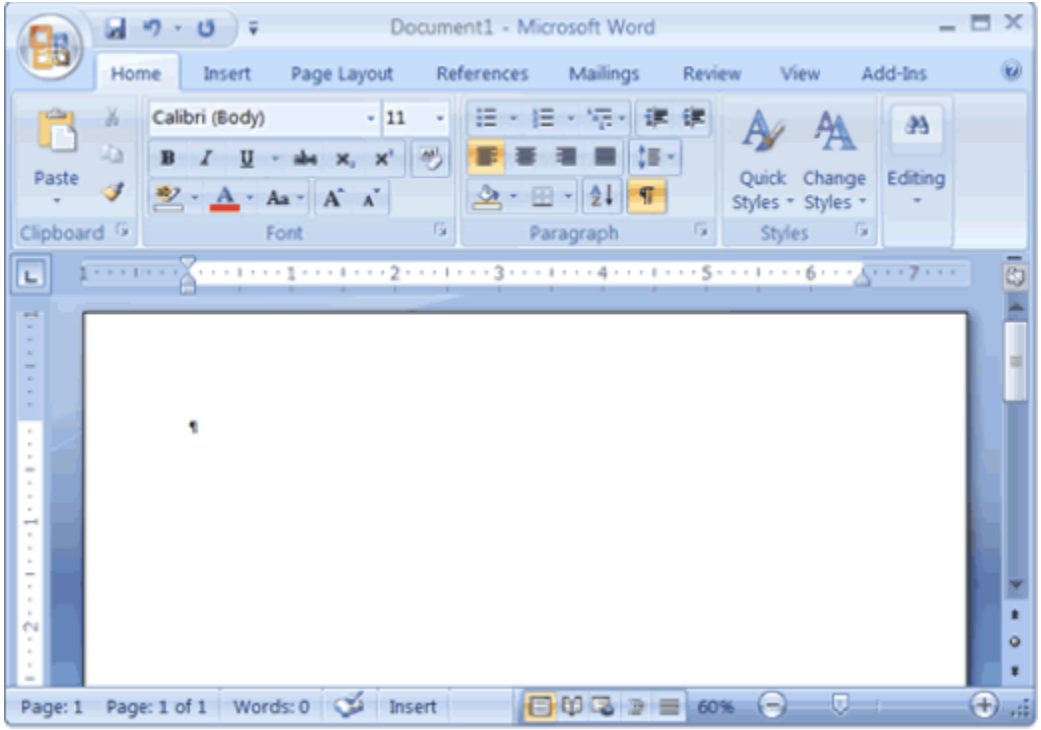
Today's Axis: The Programming Model



Why do we need event-based programming?

- Let’s look at how hard it is to write GUI code (e.g. JavaScript) in a threading world.

GUI Programming with Threading model



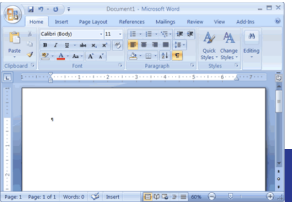

```
do {
   WaitForSomething();
   RespondToThing();
} until(forever);
```

GUI Programming

```
// Step 2: Creating the Window
hwnd = CreateWindowEx(
    WS_EX_CLIENTEDGE,
```

```
switch (message)
{
        //case WM_COMMAND:
          // handle menu selections etc.
        //break;
        //case WM_PAINT:
          // draw our window - note: you must paint something here or not trap it!
        //break;
        case WM_DESTROY:
             PostQuitMessage(0);
        break;
        default:
            // We do not want to handle this message so pass back to Windows
            // to handle it in a default way
            return DefWindowProc(hWnd, message, wParam, lParam);
}
```

```
{
    TranslateMessage(&Msg);
    DispatchMessage(&Msg);
}
```



```
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPSTR lpCmdLine, int nCmdShow)
{
    WNDCLASSEX wc;
    HWND hwnd;
    MSG Msg;

    //Step 1: Registering the Window Class
    wc.cbSize        = sizeof(WNDCLASSEX);
    wc.style         = 0;
    wc.lpfnWndProc   = WndProc;
    wc.cbClsExtra    = 0;
    wc.cbWndExtra    = 0;
    wc.hInstance     = hInstance;
    wc.hIcon         = LoadIcon(NULL, IDI_APPLICATION);
    wc.hCursor       = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);
    wc.lpszMenuName  = NULL;
    wc.lpszClassName = g_szClassName;
    wc.hIconSm       = LoadIcon(NULL, IDI_APPLICATION);

    if(!RegisterClassEx(&wc))
    {
        MessageBox(NULL, "Window Registration Failed!", "Error!",
            MB_ICONEXCLAMATION | MB_OK);
        return 0;
    }

    // Step 2: Creating the Window
    hwnd = CreateWindowEx(
        WS_EX_CLIENTEDGE,
        g_szClassName,
        "The title of my window",
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT, 240, 120,
        NULL, NULL, hInstance, NULL);

    if(hwnd == NULL)
    {
        MessageBox(NULL, "Window Creation Failed!", "Error!",
            MB_ICONEXCLAMATION | MB_OK);
        return 0;
    }

    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    // Step 3: The Message Loop
    while(GetMessage(&Msg, NULL, 0, 0) > 0)
    {
        TranslateMessage(&Msg);
        DispatchMessage(&Msg);
    }
    return Msg.wParam;
}
```

GUI Programming

```
switch (message)
{
        //case WM_COMMAND:
          // handle menu selection
        //break;
        //case WM_PAINT:
          // draw our window - not
        //break;
        case WM_DESTROY:
             PostQuitMessage(0);
        break;
        default:
            // We do not want to handle this message so pass back to Wir
            // to handle it in a default way
            return DefWindowProc(hWnd, message, wParam, lParam);
}
```

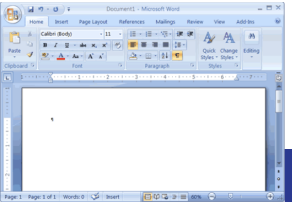
```
void OnMove() { … }
void OnSize() { … }

void OnPaint() { … }
```

Hex	Decimal	Symbolic
0000	0	WM_NULL
0001	1	WM_CREATE
0002	2	WM_DESTROY
0003	3	WM_MOVE
0005	5	WM_SIZE
0006	6	WM_ACTIVATE
0007	7	WM_SETFOCUS
0008	8	WM_KILLFOCUS
000a	10	WM_ENABLE
000b	11	WM_SETREDRAW
000c	12	WM_SETTEXT
000d	13	WM_GETTEXT
000e	14	WM_GETTEXTLENGTH
000f	15	WM_PAINT
0010	16	WM_CLOSE
0011	17	WM_QUERYENDSESSION
0012	18	WM_QUIT
0013	19	WM_QUERYOPEN
0014	20	WM_ERASEBKGND

Over 1000 last time I checked!

```
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPSTR lpCmdLine, int nCmdShow)
{
    WNDCLASSEX wc;
    HWND hwnd;
    MSG Msg;

    //Step 1: Registering the Window Class
    wc.cbSize        = sizeof(WNDCLASSEX);
```

```
    // Step 3: The Message Loop
    while(GetMessage(&Msg, NULL, 0, 0) > 0)
    {
        TranslateMessage(&Msg);
        DispatchMessage(&Msg);
    }
    return Msg.wParam;
}
```



GUI Programming Distilled

```
winmain(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_THIS: DoThis(); break;
      case WM_THAT: DoThat(); break;
      case WM_OTHERTHING: DoOtherThing(); break;
      case WM_DONE: return;
      }
  }
}
```

Pros
- Simple imperative programming
- Good fit for uni-processor

Cons
- Awkward/verbose
- **Obscures available parallelism**

GUI Programming Distilled

```
winmain(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_LONGRUNNING_CPU_HOG: HogCPU(); break;
      case WM_HIGH_LATENCY_IO: BlockForALongTime(); break;
      case WM_DO_QUICK_IMPORTANT_THING: HopeForTheBest(); break;
      }
  }
}
```

Pros
- Simple imperative programming
- Good fit for uni-processor

Cons
- Awkward/verbose
- **Obscures available parallelism**

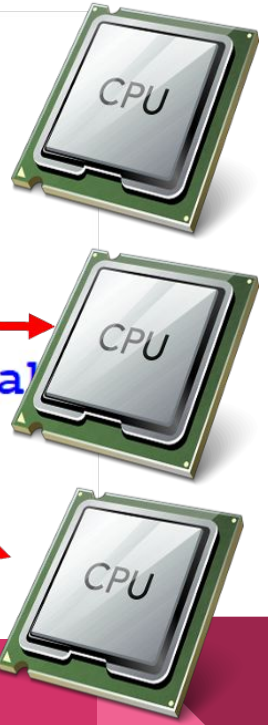
GUI Programming Distilled

```
winmain(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_THIS: DoThis(); break;
      case WM_THAT: DoThat(); break;
      case WM_OTHERTHING: DoOtherThing(); break;
      case WM_DONE: return;
      }
  }
}
```

How can we parallelize this?

Parallel GUI Implementation 1

```
winmain(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_THIS: DoThis(); break;
      case WM_THAT: DoThat(); break;
      case WM_OTHERTHING: DoOtherThing(); breal
      case WM_DONE: return;
      }
  }
}
```



Parallel GUI Implementation 1

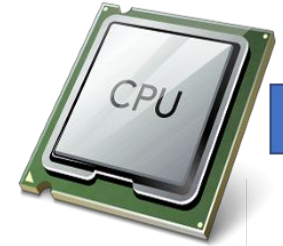
```
winmain() {
    pthread_create(&tids[i++], DoThisProc);
    pthread_create(&tids[i++], DoThatProc);
    pthread_create(&tids[i++], DoOtherThing
    for(j=0; j<i; j++)
        pthread_join(&tids[j]);
}

DoThisProc() {
    while(true)
        if(This
            DoT
    }
}
```

Pros:

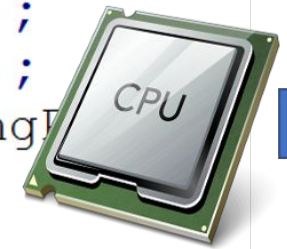
- Encapsulates parallel work

Cons:

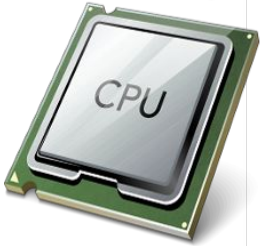
- Obliterates original code structure
- How to assign handlers->CPUs?
- Load balance?!?
- Utilization



DoThisProc



DoThatProc



OtherThing

Parallel GUI Implementation 2

```
winmain() {
    for(i=0; i<NUMPROCS; i++)
        pthread_create(&tids[i]
    for(i=0; i<NUMPROCS; i++)
        pthread_join(&tids[i]);
}
```

Pros:

- Preserves programming model
- Can recover some parallelism

Cons:

- Workers still have same problem
- How to load balance?
- Shared mutable state a problem

```
threadproc(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_THIS: DoThis();
      case WM_THAT: DoThat();
      }
  }
}
```

```
threadproc(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_THIS: DoThis();
      case WM_THAT: DoThat();
      }
  }
}
```

```
threadproc(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_THIS: DoThis();
      case WM_THAT: DoThat();
      }
  }
}
```

```
threadproc(...) {
  while(true) {
      message = GetMessage();
      switch(message) {
      case WM_THIS: DoThis();
      case WM_THAT: DoThat();
      }
  }
}
```

Extremely difficult to solve without changing the whole programming model…so **change it**!

Event-based Programming: Motivation

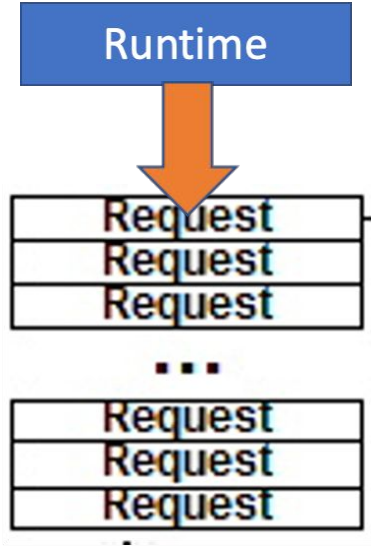
- Threads have a *lot* of down-sides:
  - Tuning parallelism for different environments
  - Load balancing/assignment brittle
  - Shared state requires locks
    - Priority inversion
    - Deadlock
    - Incorrect synchronization
  - …
- Events: *restructure programming model to have no threads!*

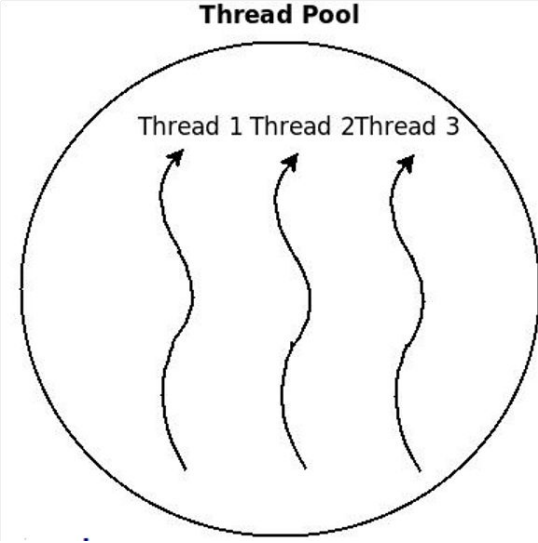
Event Programming Model Basics

- Programmer *only writes events*
  - Event: an object queued for a module (think Threadpool task)
- Basic primitives
  - create_event_queue(handler) -> event_q
  - enqueue_event(event_q, event-object)
    - Invokes handler (eventually)
  - event_wait(event-object)
- Scheduler decides which event to execute next
  - E.g. based on priority, CPU usage, etc.

Event-based Programming

```
PROGRAM MyProgram {
    OnSize() {}
    OnMove() {}
    OnClick() {}
    OnPaint() {}
}
```





Another Event-based Program

```
PROGRAM MyProgram {
    OnOpenFile() {
        char szFileName[BUFSIZE]
        InitFileName(szFileName);
        FILE file = ReadFileEx(szFileName);
        LoadFile(file);
        RedrawScreen();
    }
    OnPaint();
}
```

Uses Other Handlers! (call OnPaint?)

Burns CPU!

Blocks!

How do we gain back responsiveness? More handlers!

```
PROGRAM MyProgram {
    TASK ReadFileAsync(name, callback) {
        ReadFileSync(name);
        Call(callback);
    }
    CALLBACK FinishOpeningFile() {
        LoadFile(file);
        RedrawScreen();
    }
    OnOpenFile() {
        FILE file;
        char szName[BUFSIZE]
        InitFileName(szName);
        EnqueueTask(ReadFileAsync(szName, FinishOpeningFile));
    }
    OnPaint();
}
```

Good runtimes will spawn a new thread to run blocking I/O calls

Thread pool work must run to completion!
More, smaller work => more chances for the underlying threads to run high priority tasks!

High priority task

Continuations

```
PROGRAM MyProgram {
    OnOpenFile() {
        ReadFile(file, FinishOpeningFile);
    }
    OnFinishOpeningFile() {
        LoadFile(file, OnFinishLoadingFile);
    }
    OnFinishLoadingFile() {
        RedrawScreen();
    }
    OnPaint();
}
```

Callback

Stack Ripping

```
1  PROGRAM MyProgram {
2      TASK ReadFileAsync(name, callback) {
3          ReadFileSync(name);
4          Call(callback);
5      }
6      CALLBACK FinishOpeningFile() {
7          LoadFile(file);
8          RedrawScreen();
9      }
10     OnOpenFile() {
11         FILE file;
12         char szName[BUFSIZE];
13         InitFileName(szName);
14         EnqueueTask(ReadFileAsync(szName, FinishOpeningFile));
15     }
16     OnPaint();
17 }
```

Stack-based state out-of-scope!
Requests must carry state

Closures to the Rescue!

- Closures allow us to “capture” local variables

```
getFileName(|filename| {
    loadFile(filename, |data| {
        // do something with data
    });
});
```

Futures and Promises: closures with less parentheses

```
filename = enqueue(getFileName).wait();
data = enqueue(loadFile, filename).wait();
// do something
```

"Future"

Returns a "promise"d value

Remind you of anything?

Coroutines!

We want it to look like single-threaded programming, but to actually be event-based!

Coroutine version

```
filename = spawn(getFileName).join();
data = spawn(loadFile, filename).join();
// do something with data
```

We want it to look like single-threaded programming, but to actually be event-based!

Threads vs Events

- Thread Pros
  - Overlap I/O and computation
  - While looking sequential
  - Intermediate state on stack
  - Control flow naturally expressed
- Thread Cons
  - Synchronization required
  - Overflowable stack
  - Stack memory pressure

- Event Pros
  - Easier to create well-conditioned system
  - Easier to express dynamic change in level of parallelism
- Event Cons
  - Difficult to program
  - Control flow between callbacks obscure
  - When to deallocate memory
  - Incomplete language/tool/debugger support
  - Difficult to exploit concurrent hardware

Language-level Futures: the sweet spot?

Async/Await: Language level support for Futures

- Instead of enqueue() and wait() being *functions*, make them *keywords*
  - The compiler can recognize and optimize these points

Future: a handle representing an in-progress asynchronous function

await: wait for a future to finish

async: declares that a function returns a future, not a value

Async/Await in Rust

Rust’s Futures work a little differently:

- tokio::spawn enqueues a Future onto tokio’s run queue
- async functions do nothing unless enqueued or awaited!
  - Lazy execution, different than other languages

Async/Await in Rust

```rust
async fn page_title(url: &str) -> Option<String> {
    let response = tokio::get(url).await;
    let response_text = response.text().await;
    Html::parse(&response_text)
        .select_first("title")
        .map(|title| title.inner_html())
}
```

Yield point!

Async/Await backend options

- 1 Coroutine / 1 Thread per event
- 1 Threadpool Task per event
  - Stack reuse between each task
    - Task dependencies (awaits within async functions) can make for really big stacks…
      - Solved by Rust’s approach! (Wednesday’s lecture)

Multiprocessing vs Multithreading

- Thread (an execution context)
  - register backup space
  - interrupts enabled state
  - stack
- Process
  - a list of threads
  - an address space (aka a bunch of private memory) (containing heap & all thread stacks)
  - open files & their internal cursors
  - a creating user & group & permissions & a binary…

Multiprocessing vs Multithreading

What’s new if I try to use multiprocessing to parallelize my programs?

- synchronization must be rebuilt
  - No shared memory between processes -> cannot communicate via atomics
  - OS provides Inter Process Communication mechanisms like pipes, sockets, shared mmap regions
- larger memory footprint

Python’s GIL

Global Interpreter Lock

While interpreting user code, hold a mutex to protect against every data race.

- Multithreading no longer can give parallelism!
- Multiprocessing is required in this case

Optional Reading

https://doc.rust-lang.org/book/ch17-01-futures-and-syntax.html